

Fleet Fare System - Mastery Guide

Part 2 — A practical engineering guide to mastering fleet fares for SK Baghel Tour & Travels

Repository: github.com/tarun1sisodia/ArenaAI • Branch: main

Your goal Become able to explain every rupee in a quote, predict every boundary case, and prove that the homepage, booking desk, stored booking and payment all agree. That requires domain modeling, deterministic math, clear source-of-truth rules, and tests—not merely knowing per-kilometer rates.

1. Think like a pricing engineer

A fare system is a small financial rules engine. It takes a validated trip, selects a commercial product, resolves trustworthy inputs, applies a deterministic policy, returns a breakdown and records the exact rule version used. The frontend displays the result; the backend owns the amount.

Invariant #1 For one quote request and one fare-rule version, the same validated input must yield the same breakdown and total. Any different amount must be explainable by a changed input, product, rule version, promotion state, or explicit operator adjustment.

2. Model the business before coding

Dimension	Questions you must answer	Example for this project
Service / product	What is being sold?	Outstation route, local sightseeing, airport transfer, tour package, monument transfer.
Fleet tier	Which priced vehicle class?	Sedan, Ertiga, Innova Crysta, Tempo Traveller, Force Urbania.
Trip semantics	One-way, round-trip, hourly/local, or multi-day?	Agra → Delhi one-way differs from Agra local 8hr/80km.
Distance basis	Which distance can be charged?	Trusted route km, agreed corridor km, included local km, or explicit commercial fixed fare.
Price model	Per-km, fixed-by-fleet, or fixed starting price plus upgrade?	Routes may be per-km; a curated Taj tour may have a fixed price per fleet.
Included usage	Included km/hours/nights?	8 hours / 80 km included; define extra km, extra hour, overstay and parking separately.
Commercial add-ons	What is included vs extra?	Toll, parking, state tax/permit, driver allowance, night halt, waiting.
Discount / deposit	Which subtotal may be discounted?	Promo on permitted subtotal; advance from final payable total.
Lifecycle	When does a price become binding?	Displayed estimate → server quote → booking snapshot → payment order →

Dimension	Questions you must answer	Example for this project
		confirmed booking.

3. Keep the main service types separate

Fare mode	Conceptual formula	Rules to decide explicitly
Route / outstation per-km	$\text{base} = \text{billable_km} \times \text{fleet_rate}$	Distance source, one-way vs return distance, minimum billed km per day, toll/permit, driver allowance.
Fixed corridor price	$\text{base} = \text{published fleet-specific corridor amount}$	Whether route price overrides per-km rate; how updates to rates affect the corridor; return fare policy.
Fixed tour package	$\text{base} = \text{package.fleetPrices}[\text{fleet}]$	Package is commercial inventory; changing fleet per-km rate should not silently change package price.
Starting price + fleet upgrade	$\text{base} = \text{package.startingPrice} + \text{upgrade}[\text{fleet}]$	Use only when no explicit fleet price exists; document fallback order and prevent a missing price becoming free/incorrect.
Local package	$\text{base} = \text{fixed package price for included km/hours}$	Extra km, extra hour, package day rollover, waiting/parking, included stops.
Airport / station transfer	$\text{base} = \text{transfer tariff or explicit route fare}$	Match exact transfer identity, pickup/drop direction, after-hours fee, toll inclusion.
Commercial group exception	$\text{base} = \text{rule-specific billed km} \times \text{tier rate}$	300 km boundary, forced return convention, trip days, driver allowance and whether user-selected trip type can override.

Non-negotiable modeling rule A “route”, “package”, “local package”, and “transfer” are different commercial products. Do not make one ambiguous function guess the business model from a slug, text label, distance, or overlapping boolean flags.

4. Build a quote pipeline with clean responsibilities

A. Parse and validate — Validate canonical fleet code, trip type, origin/destination identity, pickup/return timestamps, passenger count, product status, promo format, and allowed options. Reject unknown types rather than silently mapping them to Sedan.

B. Resolve product — Look up the selected route/package/local tour/transfer by canonical ID or stable code. Verify published + active status. Never trust a product name, price, or distance submitted by the browser.

C. Resolve distance and time — Use the authoritative route record or a server-side routing/geocoding provider. Store the source and version. Separate one-way road distance, billable distance, actual return distance, and minimum billed km.

D. Resolve rule version — Load a single active fare ruleset and keep its version with the calculation. Every rule should have explicit precedence and effective dates.

E. Run the pure fare engine — Use a calculator chosen by product type and vehicle class. No database, network, or React dependencies inside the core arithmetic.

F. Apply shared charges consistently — Apply driver/night/toll/permit/waiting/extra-hour charges and promo eligibility once in a declared order.

G. Finalize money — Round only at documented points. Calculate final total, advance and balance, validate nonnegative values and return a line-item breakdown.

H. Persist the quote snapshot — For a booking, persist all quote inputs, outputs, price-source identifiers, fare version, timestamps, and breakdown. Never recalculate old booking totals just because the current rule changed.

I. Bind payment to snapshot — Create payment amount from the persisted booking balance/advance, not from client fields. Use idempotency and verify webhook authenticity before changing payment state.

5. Define fare math with zero ambiguity

5.1 Start with a line-item equation

A general, explainable quote can be modeled as:

Subtotal = Base fare + Driver allowance + Night/Halt charges + Toll/Permit + Other approved fees

Total = max(0, Subtotal – Eligible discount)

Balance = Total – Advance

The exact tax/fee treatment, cap/floor, rounding and payment policy must be agreed commercially. If GST or other taxes apply, model them as explicit line items rather than silently burying them in a rate.

5.2 Separate actual, billed, included and minimum distance

Term	Definition	Example
Actual route distance	Trusted one-way road distance returned by the route source.	Agra to destination = 178 km.
Billed distance	Kilometers actually used by the selected fare formula.	Tempo/Urbania below 300 km: $178 \times 2 = 356$ km under current rule.
Included kilometers	Kilometers included in the fixed local package price.	8hr/80km local package includes 80 km.
Excess distance	Distance beyond included allowance charged by a published extra rate.	Actual trip 96 km => 16 excess km if the package terms say so.
Minimum daily km	The minimum billable distance floor for eligible outstation days.	Two-day minimum at 300 km/day means 600 billed km before comparing actual-distance formula.
Round-trip multiplier	A commercial multiplier applied to a basis fare; not always equivalent to distance $\times$ 2.	Standard same-day round trip uses $1.85\times$ per current implementation; group fleet threshold is a separate rule.

5.3 Use explicit units and money handling

- Name variables with units: distanceKm, billableKm, rateInrPerKm, amountInr, amountPaise, durationMinutes. Do not call different values simply distance or amount.
- Use integer paise or a reliable decimal representation for money at persistence/payment boundaries. In-memory rupee calculations may be acceptable only with a documented integer-rounding policy.

- Decide whether to round each line item, the subtotal, the percentage discount, or only final total. Avoid mixing Math.round, round-to-₹100 and unrounded floats across service branches.
- Validate every numeric input with Number.isFinite, sign/range checks, and integer checks where required. Reject NaN, Infinity, negative values and extreme limits.
- Keep displayed rates and payable totals distinct: a ₹/km rate is not a quote until the distance, minimums, trip mode and surcharges are known.

6. Fleet policies: translate the code into a business table

Fleet tier	Default rate observed	Seats observed	Current special treatment
Sedan	₹10/km	4	Standard strategy; default cab driver/night allowance behavior.
Ertiga	₹14/km	6	Standard strategy.
Innova Crysta	₹18/km	6	Standard strategy.
Tempo Traveller	₹25/km	12	Group/commercial exception: force round trip for distance <300 km; ₹500/day driver allowance default; promo restricted by default.
Force Urbania	₹34/km	16	Group/commercial exception: same threshold and allowance behavior as Tempo.

These are code defaults. Before treating them as current business policy, inspect the active production fare_rules row, any product-level fleetPrices and the operator-approved fare sheet.

7. Boundary conditions you must own

Case	Why it matters	Expected test assertion
Distance = 0 / negative / NaN / Infinity	Can create free, negative, or invalid prices.	Request rejected before fare calculation.
Distance = 299, 299.9, 300, 300.1 km	Group vehicle behavior flips at 300 km.	199? No: test exact 299/299.9 doubles; 300/300.1 does not force return under this rule.
Round trip with same-day return	Multiplier and calendar-day calculation can interact.	Expected billed basis, applied multiplier flag, driver allowance and total are consistent.
Multi-day trip crossing midnight / IST day boundary	Duration in hours is not necessarily commercial calendar days.	IST calendar-day policy produces an explicit day count; no UTC off-by-one.
Missing return datetime	Some strategies double distance conditionally; others use day count defaults.	Validation/defined policy decides whether round trip is allowed without a return date.
Origin equals destination	Can accidentally select local sightseeing logic.	Only a valid local-tour selection can enter local pricing; outstation selection is rejected.
Unknown origin/destination	Fallback map can create plausible but inaccurate estimated distance.	Quote is marked indicative or requires server routing; never silently treat arbitrary fallback as guaranteed.
Unknown vehicle tier	Legacy helper can fall back to Sedan.	API rejects unknown tier with structured validation error.

Case	Why it matters	Expected test assertion
Vehicle inactive in admin	A rate can exist even when vehicle is not bookable.	Quote returns vehicle unavailable; frontend cannot bypass it.
Package without fleet price	Can fall through to an unintended price model.	Use documented upgrade fallback or reject with TIER_NOT_PRICED.
Promo expires / max redemption hit / two simultaneous bookings	Validation races and eligibility vary by fleet.	Invalid promo does not discount; redemption cap is enforced transactionally and idempotently.
Promo + toll + night + driver fee	Subtotal order affects final amount.	Breakdown, discount base and total all reconcile exactly.
Pickup at night boundary: 19:59 / 20:00 / 05:59 / 06:00	Engine and comments currently disagree about window.	One central policy determines outcomes, tested at exact boundary.
Fare-rule update during checkout	Quote can go stale between displayed quote and draft.	Draft either honors still-valid quote version or returns quote-expired and asks customer to refresh.
Admin changes rates after booking	Customer should not have historical total changed.	Stored snapshot and payment amount remain immutable.
Frontend cache/static HTML is stale	The page can advertise old amounts after admin update.	Version invalidation / revalidation works and UI labels nonbinding editorial prices honestly.

8. Product modeling and database design

Prefer explicit product-specific records over a giant generic JSON object with overlapping flags. Exact schema can evolve, but each field needs an owner and one meaning.

Entity	Fields to define	Invariant
FleetTier	code, displayName, seats, luggageCapacity, active, defaultPerKmRate, effective dates	Canonical code is stable; alias/name changes do not change identity.
FareRuleVersion	version, status, effectiveFrom, createdBy, approvedBy, ruleConfigHash	At most one active version; activation is atomic and auditable.
RouteProduct	canonical route ID, origin/destination IDs, trusted distance, duration, fare model, toll policy, status	A route row describes corridor facts; it does not accidentally contain a second hidden price authority.
TourPackage	packageCode, included days/nights, fleetPrices, upgrade policy, inclusions/exclusions, status	Fixed commercial price is isolated from per-km rate changes unless explicitly configured otherwise.
LocalPackage	durationHours, includedKm, fleet fixed prices, excess km/hour rate, allowed stops, charges	Included capacity and overage policy are explicit and consistently enforced.
TransferProduct	origin/destination/airport IDs, direction, fixed fleet prices or per-km model, included toll terms	Matched by canonical endpoints/product ID, not loose substring search.
Promo	code, active window, min subtotal, discount type/value, cap, fleet/product eligibility, max redemptions	Eligibility is deterministic; redemption count is concurrency-safe.
QuoteSnapshot / Booking	input snapshot, breakdown, product IDs, distance source, rule version, totals, expiry, currency	Persisted total is authoritative for booking/payment and historical records do not mutate.

9. Admin fare editor: safe rules management

- Edit the rule draft; never overwrite the only active ruleset in place. Show before/after diffs per tier and per policy.
- Validate ranges: rate > 0 and commercially reasonable, seats positive integer, allowance >= 0, times 0–23, multiplier within a policy-approved range, and supported tiers only.
- Require explicit confirmation for changes that materially alter customer totals, e.g. 300 km threshold, rate changes, promo eligibility, or toll-inclusive status.
- Run a quote preview matrix before activation: all five fleets × representative routes × one-way/round-trip/multi-day/local/package/transfer modes.
- Activate version transactionally, record actor/reason/time, and retain rollback to last-known-good.
- After activation, check public fleet and quote APIs, bust/version catalog cache, refresh build-time/static price displays, then show actual propagation state in admin.
- Never display “synchronized across customer apps” until the live customer surfaces have been verified against the new version.

10. API contracts: the clean boundary

The repo currently exposes POST /api/v1/fares/calculate and GET /api/v1/fleet. The repo’s own audit proposes explicit route and package quote endpoints. Separate endpoints are useful if they prevent the server from confusing a fixed package with a route calculation; shared engine code should still be reused.

Conceptual route request

```
POST /api/v1/quotes/route
{ routeId OR originId + destinationId, tripType, fleetCode, pickupDatetime, returnDatetime?, passengerCount, promoCode? }
```

Conceptual package request

```
POST /api/v1/quotes/package
{ packageCode, fleetCode, pickupDatetime, passengerCount, promoCode? }
```

Conceptual quote response

```
quoteId, expiresAt, fareVersion, productId, productType, fleetCode, currency, distanceKm, billedKm, pricingType, lineItems[], subtotal, discount, totalFare, advanceAmount, balanceAmount, ruleTrace[]
```

Do not accept totalFare/advanceAmount from the browser. Prefer canonical IDs over display strings. If approximate address-to-address quotes are allowed, return a distanceSource + estimate status and define when the quote becomes binding.

11. Testing like a pricing engineer

11.1 Core test layers

- Unit tests: each calculator, strategy, night-window function, promo eligibility and advance/rounding helper.
- Contract tests: request validation, unknown tiers, strict schemas, error envelopes, output shape and currency units.
- Property tests: total never negative; advance ≤ total; balance = total – advance; billed distance never negative; identical normalized input + rule version yields identical output.
- Combinatorial tests: each service mode × each fleet × dates × distance boundaries × promo allowed/disallowed × toll/night extras.

- Integration tests: admin update → database active version → public fleet endpoint → quote response version → booking fare snapshot → payment order amount.
- Presentation parity tests: homepage/routes/packages/monuments and booking quote show the exact same server-provided price or are clearly labeled as editorial starting prices.
- Historical immutability: after active rules update, existing booking total, balance, and checkout binding remain unchanged.

11.2 Worked examples using the observed defaults

These examples demonstrate rules, not a promise that current production quotes equal these amounts. Assume default rates and no toll/night fee/promo unless listed. The exact result depends on which branch the request takes and whether a route record contributes static fares.

Scenario	Calculation under the named rule	Result / lesson
Tempo Traveller, 120 km, group exception	Billable distance = $120 \times 2 = 240$ km; base = $240 \times ₹25 = ₹6,000$; add ₹500 driver allowance for one day.	₹6,500 subtotal before eligible extras. The 300-km rule changes the billable km, not the rate.
Urbania, 299.9 km, group exception	Billable distance = $299.9 \times 2 = 599.8$ km; base = $599.8 \times ₹34 = ₹20,393.2$, then rupee rounding per calculator; add ₹500/day.	About ₹20,893 before extras. Decimal-km rounding policy must be explicit.
Tempo Traveller, 300 km, requested one-way	Threshold condition 'distance < 300' is false; billable distance follows one-way input, so $300 \times ₹25 = ₹7,500$; add ₹500/day.	₹8,000 before extras. A 0.1 km increase from 299.9 can cause a large fare discontinuity; that is a business choice requiring explicit display/approval.
Sedan, one-way 210 km with custom rate	If custom DB rate ₹10/km is applied, base = $210 \times ₹10 = ₹2,100$.	But if the route has static catalogue fares and rate is not flagged custom, static route amount may be used instead. This is why source-of-truth precedence must be fixed.
Standard cab, one-day round trip, 210 km	At default ₹10/km: route/multiplier basis and minimum floor both participate. Minimum = $300 \times ₹10 = ₹3,000$; route fare $\times 1.85$ may be higher.	The engine takes the larger calculated base. Precisely test the route fare, rate override and return-date path.
8hr/80km local Sedan	Curated package fixed price = ₹1,900; not ' $80 \times ₹10$ '.	Local package price is a different product model, not a route per-km quote.

12. Security and abuse cases

- Distance tampering: customer sends 1 km for a long route. Server recomputes distance from canonical route/provider.
- Rate tampering: customer sends perKm = 1 or negative fare. Unknown money fields are rejected or ignored; totals are not accepted.
- Fleet tampering: unknown vehicle enum or inactive tier. Return a validation/unavailable error; never default to Sedan.
- Product tampering: change package id/slug/status after quote. Resolve on server and bind to canonical published product ID/version.
- Promo abuse: brute force code, race maximum redemptions, reuse promo across competing bookings. Rate limit validation and transactionally reserve use.
- Quote replay: quote expired or fare version no longer valid. Either honor its defined validity window or require a refreshed quote; do not silently change price after user has accepted.
- Payment tampering: browser reports a lower payment amount or duplicate create-checkout request. Derive amount from persisted booking and use idempotency.

13. Architecture pattern to aim for

- HTTP/controller: parse and authenticate; never calculate fare.
- Application service: orchestrates product lookup, distance source, active rule version, promo context and persistence.
- Domain calculator/strategy: pure deterministic fare math; no DB, network, browser or environment reads.
- Rule modules: small named functions for outstation distance minimum, group-vehicle threshold, night window, promo eligibility, money rounding, cancellation/refund policy.
- Finalizer: shared line item order and response creation; a single path so route/package/transfer promos and charges do not drift.
- Repositories: database access and transactions, one active fare version, product published status and atomic promo redemption.
- Frontend: request quote, display server breakdown and amount; format currencies and labels only. An “indicative” mode must be explicit and nonbinding.
- Observability: log quote ID, rule version, product, fleet, distance source and rule trace—never log secrets or unnecessary personal data.

14. Suggested 4-week mastery track

Week	Study / build	You should be able to demonstrate
1 — Trace	Read fare.service, fare.engine, product/distance resolvers, strategy, calculators, finalizer. Trace three real example requests manually.	Explain exactly which branch runs and where every line item comes from.
2 — Rules and tests	Write table-driven tests for all fleets at 299/299.9/300/300.1 km, one-way/round-trip, single/multi-day, night boundaries, promo and fixed packages.	Predict totals before running tests; identify inconsistent rules with clear reproductions.
3 — Source of truth	Inventory every displayed price. Define fixed-package vs per-km contracts. Move display surfaces to server quote responses; preserve SEO with honest starting-price labels and revalidation.	No customer page invents a payable amount or contradicts the quote for the same inputs.
4 — Production-grade	Add rule versioning, transactional activation, snapshots, promo redemption safety, payment binding, quote expiry and audit logs. Run the full integration matrix.	Show a trace from admin edit to new quote to immutable booking/payment, plus rollback and regression evidence.

15. Final mastery checklist

- [] I can name the product type and price model before calculating anything.
- [] I can identify distance source, actual distance, billed distance and any minimum floor.
- [] I can explain the exact meaning of the 300 km boundary for each product/fleet combination.
- [] I know which charges apply, the eligibility conditions, and the order in which they are applied.
- [] I can calculate advance, balance and promo outcomes from the server’s exact rounding policy.
- [] I can prove every public price comes from a canonical source or is clearly labelled nonbinding editorial content.
- [] A fare-rule change is versioned, reviewed, audited, activated atomically and propagated to cache/static surfaces.
- [] A prior booking remains unchanged after an admin rule change.

[] Payment amount comes from the persisted server quote/snapshot, never the browser.

[] Every meaningful boundary and invalid input has an automated regression test.

16. Repository references

- Main backend fare service: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/fare.service.ts>
- Core fare engine: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/fare.engine.ts>
- Strategy selection: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/fare.strategy.ts>
- Standard calculator: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/calculators/route.calculator.ts>
- Group commercial calculator: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/calculators/commercial-group.calculator.ts>
- Finalizer: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/calculators/fare.finalizer.ts>
- Admin fare editor: <https://github.com/tarun1sisodia/ArenaAI/blob/main/admin/src/pages/FaresPage.tsx>
- Customer price matrix: https://github.com/tarun1sisodia/ArenaAI/blob/main/docs/PRICE_SOURCE_MATRIX.md
- Fleet migration plan: https://github.com/tarun1sisodia/ArenaAI/blob/main/docs/MIGRATION_PLAN.md
- Fare implementation plan:
https://github.com/tarun1sisodia/ArenaAI/blob/main/docs/FLEET_PRICING_MASTER_IMPLEMENTATION_PLAN.md

17. Worked Scenario Book: What, When, How, and Why

Use this chapter like a walkthrough workbook. For every quote, answer four questions before doing the arithmetic: (1) What product is this? (2) Which price model applies? (3) Which branch or threshold activates? (4) Which extra line items are added before discount and deposit? The figures below use the repository defaults unless stated otherwise. Where current code behavior is inconsistent, it is called out explicitly instead of being presented as ideal behavior.

The one equation to remember

Subtotal = base fare + driver allowance + night allowance + tolls/other authorized charges.

Total = max(allowed minimum, subtotal – valid discount) according to the selected product policy.

Balance = total – advance. The backend must calculate these values; the frontend only displays them.

Example 1 — Sedan, 180 km, one-way: route fare vs per-kilometer fare

Situation: A customer asks for a one-way route of 180 km in a Sedan. The default Sedan rate is ₹10/km. Assume there is no night fee, driver allowance, toll, or promo.

Step	Canonical calculation / question	Result
1. Product	Outstation route; one-way; Sedan.	Use route pricing rules, not a local-tour or package rule.
2. Pure distance math	$180 \text{ km} \times ₹10/\text{km}$.	₹1,800
3. Minimum floor?	The fallback route estimator has a Sedan floor of ₹2,200. Is that floor part of this product policy?	If enabled: $\max(₹1,800, ₹2,200) = ₹2,200$.
4. Current-code caution	Without an active vehicle override, the strategy can use the route catalogue fare. With an override, it can use $\text{distance} \times \text{rate}$. The rate-driven branch does not currently apply the same minimum floor.	Same trip can differ by source/branch; normalize the rule before production.

Why it matters: a per-kilometer rate alone does not define the fare. You must also state whether a minimum fare applies, which products it applies to, and whether static route prices override the formula.

Example 2 — Tempo Traveller just below and at the 300 km boundary

Observed rule: Tempo Traveller and Urbania use doubled distance when the origin-to-destination distance is less than 300 km. At 300 km and above, an explicitly one-way trip is billed one-way. The Tempo rate is ₹25/km and the default driver allowance is ₹500/day. Assume one commercial day and no other charges.

Distance / request	Billable km	Base fare	Driver allowance	Subtotal
299 km, Tempo, one-way	$299 \times 2 = 598$	$598 \times ₹25 = ₹14,950$	₹500	₹15,450
299.9 km, Tempo, one-way	$299.9 \times 2 = 599.8$	$599.8 \times ₹25 = ₹14,995$	₹500	₹15,495
300 km, Tempo, one-way	300	$300 \times ₹25 = ₹7,500$	₹500	₹8,000
300.1 km, Tempo, one-way	300.1	₹7,502.50, rounded to ₹7,503	₹500	₹8,003

Important business boundary

The current rule creates a sharp price drop when the route crosses from 299.9 km to 300 km. That is not a rounding error; it follows directly from the commercial rule. The business owner must explicitly approve this behavior and customer-facing wording. Consider whether the threshold should be based on verified route distance, a different pricing tier, or a continuous minimum-fare formula.

Why it matters: test 299, 299.9, 300, and 300.1 km. Testing only 100 km and 500 km will miss the most consequential boundary defect. Also test a 300–499 km round trip with and without returnDatetime, because the current commercial strategy has a special doubling condition when returnDatetime is absent.

Example 3 — Urbania at the same boundary

Urbania is ₹34/km, plus the default ₹500/day driver allowance. The same 300 km rule yields:

Distance / request	Billable km	Base fare (rounded)	Driver allowance	Subtotal
299 km, one-way request	598	$598 \times ₹34 = ₹20,332$	₹500	₹20,832
299.9 km, one-way request	599.8	$₹20,393.20 \rightarrow ₹20,393$	₹500	₹20,893
300 km, one-way request	300	$300 \times ₹34 = ₹10,200$	₹500	₹10,700
300.1 km, one-way request	300.1	$₹10,203.40 \rightarrow ₹10,203$	₹500	₹10,703

Question to ask: Is the trip type shown clearly to the customer after the system forces a round trip? The UI must never say “one-way fare” while pricing twice the distance.

Example 4 — Sedan same-day round trip: why the route source matters

Situation: 210 km, same-day round trip, Sedan ₹10/km. Suppose a static route catalogue supplies a one-way fare of ₹2,500. Current strategy calculates a same-day multiplier of 1.85× and a 300 km/day minimum floor.

Calculation	Math	Current branch result
Static one-way base	Given by route catalogue	₹2,500
Same-day round multiplier	$₹2,500 \times 1.85$	₹4,625
Minimum daily floor	$300 \text{ km} \times ₹10$	₹3,000
Chosen base fare	$\max(₹4,625, ₹3,000)$	₹4,625
Driver allowance	Same-day standard cab branch	₹0

Compare the formula-driven alternative: if base one-way fare were purely $210 \times ₹10 = ₹2,100$, then $1.85 \times ₹2,100 = ₹3,885$ and $\max(₹3,885, ₹3,000) = ₹3,885$. The ₹740 difference shows why “route catalogue price” versus “distance × rate” must be an explicit fare model, not an accidental implementation detail.

Example 5 — Multi-day Sedan: minimum kilometers per day

Situation: a two-calendar-day trip, 150 km corridor, Sedan ₹10/km, default minimum 300 km/day, and ₹300/day driver allowance for multi-day standard-cab trips. Assume no night charge, toll, or promo.

Line item	Calculation	Amount
Minimum billable distance	$300 \text{ km/day} \times 2 \text{ days}$	600 km
Minimum distance base	$600 \text{ km} \times ₹10/\text{km}$	₹6,000
Actual-route comparison	The engine also computes an actual-distance-based candidate; the minimum floor wins here.	₹6,000 base fare
Driver allowance	$₹300/\text{day} \times 2 \text{ days}$	₹600
Subtotal before extras	$₹6,000 + ₹600$	₹6,600

Implementation check

The current strategy returns the original route distance as `billedKm` for this branch even when the minimum-distance fare uses 600 billable km. The breakdown should report both actualDistanceKm = 150 and billedKm = 600, so customers and support staff can reconcile the quote.

Example 6 — Local sightseeing: do not confuse included kilometers with per-km pricing

Situation: Agra Local Sightseeing, 8 hours / 80 km, Sedan. The curated local package has a fixed Sedan price of ₹1,900.

Wrong mental model	Correct model for this product
80 km × ₹10/km = ₹800, therefore quote ₹800.	Use the configured fixed package fare: ₹1,900.
Treat the “80 km” as an automatic formula.	Treat it as included capacity/terms unless a separately approved extra-km policy is configured.
Invent a surcharge if the customer travels 90 km.	Define and publish an explicit excess-km and/or excess-hour rule first; the current curated local calculator does not automatically prove such a charge is applied.

Why: local sightseeing sells a time-and-distance package, not necessarily a simple distance product. Store included hours, included distance, excess distance rate, excess-hour rate, and permitted stops as explicit fields if the business wants overage charging.

Example 7 — Fixed tour package with a fleet-specific price

Suppose an active, published tour package has `fleetPrices = { sedan: 5200, ertiga: 6100 }`. A customer chooses Ertiga. Global Ertiga per-km rate is ₹14/km.

Step	Calculation	Result
Select package tier price	Read `fleetPrices.ertiga`	₹6,100
Do not calculate route distance × rate	₹14/km is a route-rate rule, not this package price.	Still ₹6,100 base
If tier price is missing	Use the configured starting-price + upgrade policy only if the product contract explicitly permits that fallback; otherwise return a clear unpriced-tier error.	Never silently invent a price

Why: changing the global Ertiga rate should not silently change a fixed tour price. The package price changes only when its own commercial configuration changes.

Example 8 — “What if I add toll + driver + night charge + promo?”

Use a Tempo Traveller at 300 km, requested one-way. Assume the applicable rule allows the admin-configured flat toll, a night pickup is in the approved window, and a valid promo is explicitly permitted for this vehicle.

Line item	Calculation	Amount
Base fare	300 km × ₹25/km	₹7,500
Driver allowance	₹500/day × 1 day	₹500
Night allowance	Tempo night allowance	₹500
Toll	Configured route toll	₹400
Subtotal before promo	7,500 + 500 + 500 + 400	₹8,900
Valid flat discount	Assume ₹1,000 discount; promo active, dates valid, minimum met, redemption available and group-vehicle permission enabled.	–₹1,000

Line item	Calculation	Amount
Final total	8,900 – 1,000	₹7,900
Advance	28% × 7,900 = 2,212; nearest ₹100 = ₹2,200 (minimum ₹500; capped at total).	₹2,200
Balance	7,900 – 2,200	₹5,700

Addition order matters

First determine the correct product base fare. Add eligible fees (driver, night, toll, permits) according to the applicable terms. Validate promo eligibility and calculate the discount base. Then calculate total, advance, and balance. Do not calculate the advance from the pre-discount amount. Do not add a charge merely because a field exists; the selected product policy must authorize it.

Example 9 — Night allowance boundary (IST)

Pickup time in IST	Expected result under current helper default	Reason
19:59	No night allowance	Before 20:00
20:00	Apply configured tier allowance if this branch permits night charges	Inclusive start boundary
05:59	Apply configured tier allowance if this branch permits night charges	Still before 06:00
06:00	No night allowance	Exclusive end boundary

The current `isNightPickup` helper defaults to 20:00–06:00 IST, while a catalogue comment describes 22:00–05:00. Resolve this policy conflict before editing code. Also confirm whether the selected service type permits a night fee: a local package and a normal outstation route do not necessarily share the same fee policy.

Example 10 — Customer tampers with distance or total

Suppose an Agra-to-Delhi quote request contains `distanceKm: 40`, although the canonical route record says 210 km, and also contains `totalFare: 100`. Expected production behavior: the request schema should not allow a customer to determine authoritative distance or money; the server must resolve the route/product, trusted distance source, active fare version, and final fare independently.

Input from browser	Trust decision	Expected server behavior
distanceKm = 40	Untrusted for public fare calculation	Ignore/reject it and resolve the canonical route distance; return the source used.
totalFare = 100	Never authoritative	Reject the unknown field at schema validation; always recompute the total.
vehicleTier = unknown	Invalid enum	Return structured validation error; do not silently price as Sedan.
Vehicle is inactive in admin	Bookability rule	Return vehicle-unavailable; do not quote an inactive tier.

Current-code finding

`resolveTripDistanceKm` currently checks a positive request `distanceKm` before product and catalog distances, while the request schema still accepts this field for legacy compatibility. Some downstream branches clamp against catalogue distance, but this is not a safe global guarantee. Remove the field from public authority or ensure only a trusted server-derived value can reach the engine.

Example 11 — Admin changes the fleet rate after a booking

A customer confirms a Sedan quote for ₹4,500. The booking stores total ₹4,500, the calculated advance/balance, a full fare snapshot, and the rule version. The next day, admin changes Sedan from ₹10/km to ₹12/km and activates a new rule version.

Object	Should change?	Why
Future quotes	Yes, after active version takes effect	New bookings use the new published rule version.
Customer pages showing live route prices	Yes, after cache/revalidation policy completes	They must not advertise stale binding prices.
Existing booking fare snapshot	No	A confirmed/created booking is a historical commercial agreement.
Existing booking payment amount	No	Checkout amount comes from the persisted booking advance, not the newly active rate.
Audit/version log	Yes, append new event/version	Changes must be attributable and reversible.

17.12 A repeatable checklist for any “add this + this” change

- Name the product type: outstation, round trip, local package, transfer, or fixed-price tour package.
- Name the price model: fixed fleet price, per-kilometer rate, starting price + tier upgrade, or a formally defined hybrid.
- Declare the trusted distance/time source and whether the price uses actual km, billable km, included km, or minimum km/day.
- Write every line item and its condition: driver, night, toll, permit, extra hour/km, promo, tax if applicable.
- State the order of operations: base -> authorized extras -> eligible discount -> final total -> advance -> balance.
- Define boundaries and invalid inputs: 299.9/300/300.1 km, date transitions in IST, missing return date, inactive fleet, missing package tier price.
- Persist rule version and fare snapshot; never recalculate a historical booking when admin changes current rules.
- Add a happy-path test, boundary test, negative test, concurrency/integration test, and customer-display parity test.

Mastery test

You understand a fare rule when you can predict the itemized result before running the code, identify the exact function that owns each line item, explain why each rule applies, and prove the same amount travels from quote to booking to payment.